SHADOWFOX | MACHINE LEARNING ENGINEER INTERNSHIP

Intermediate Level Project Report

Car Selling Price Prediction

Predicting used-vehicle selling prices with regression models, from exploratory analysis to a saved Gradient Boosting model

Submitted by	Samarth Jain
Institute	Sage University, Indore
Internship role	Machine Learning Engineer Intern
Organization	ShadowFox (Virtual Internship Program)
Program ID	SF-PID-2026-W7613
Internship period	1 November 2026 – 30 November 2026
Project level	Intermediate
Report date	November 2026

Contents

Abstract
1. Introduction
2. Problem Statement
3. Project Objectives
4. Scope of the Project
5. Dataset Description
6. Tools and Technologies
7. Methodology and Workflow
8. Data Preprocessing
9. Exploratory Data Analysis
10. Model Development and Implementation
11. Hyperparameter Tuning
12. Model Evaluation and Results
13. Feature Importance Analysis
14. Challenges Faced and Solutions
15. Results and Discussion
16. Limitations
17. Future Improvements
18. Conclusion
19. Reproducibility and Saved Model
References

Abstract

This report documents a supervised machine-learning project that predicts the selling price of used vehicles from the ShadowFox Machine Learning Engineer Internship (Intermediate level). The dataset contains 301 listings with nine columns; after removing two exact duplicates, 299 records were used. A new feature, Car_Age, was derived from the manufacturing year, the high-cardinality Car_Name column was excluded, the three categorical features were one-hot encoded, and the data was split 80/20 into 239 training and 60 test records.

Five regression algorithms were trained as baselines (Linear Regression, Decision Tree, Random Forest, Gradient Boosting and Support Vector Regression). Random Forest and Gradient Boosting were then tuned with RandomizedSearchCV (50 sampled combinations, five-fold cross-validation). On the held-out test set the baseline Gradient Boosting Regressor performed best, with MAE 1.1349, RMSE 2.5232 and R^2 0.7530; Linear Regression was practically tied on R^2 (0.7528). Feature importance showed that Present_Price accounts for about 89.85% of the model's importance, followed by Car_Age (about 7.49%). The report also records the limitations of the study, including the small sample, the exploratory use of the test set during model comparison and the mixed composition of the dataset.

1. Introduction

Buying and selling used vehicles requires an estimate of a fair price. That price depends on many factors, such as what the vehicle would cost new today, how old it is, how far it has been driven, its fuel type, who is selling it and its transmission. Machine learning can learn these relationships from historical listings and produce price estimates for new listings.

This project applies regression techniques to a dataset of used-vehicle listings. It follows a complete workflow: data inspection and cleaning, exploratory analysis, feature engineering, training and comparing several regression models, hyperparameter tuning, evaluation with standard regression metrics, feature-importance analysis and saving the final model for reuse.

2. Problem Statement

Given the characteristics of a used vehicle (current ex-showroom price, kilometres driven, number of previous owners, age, fuel type, seller type and transmission), predict its selling price. Because the target, Selling_Price, is a continuous value, the task is a supervised regression problem.

3. Project Objectives

- Analyse the used-vehicle dataset and understand its features.
- Perform data cleaning and preprocessing.
- Conduct Exploratory Data Analysis (EDA) with distribution, relationship and correlation plots.
- Perform feature engineering and feature selection.
- Train and compare multiple regression models.
- Apply hyperparameter tuning using RandomizedSearchCV.

- Evaluate the models using MAE, MSE, RMSE and R^2.
- Identify the features that most influence the selling price.

4. Scope of the Project

The project covers a single tabular dataset and a notebook-based workflow in Python. It includes data analysis, model training, tuning and evaluation, and it saves the final preprocessor and model to a file for reuse. It does not include a deployed web application or API, time-series price forecasting, or data collection from live marketplaces. The project is therefore documented as a notebook and model-file deliverable rather than an application, and no application screenshots are included.

5. Dataset Description

The dataset (car.csv) contains 301 rows and 9 columns, with no missing values. Each row describes one listing. Table 1 lists the columns.

Table 1: Dataset columns

Column	Type	Description
Car_Name	Text	Name of the vehicle (98 unique names after de-duplication)
Year	Integer	Year of manufacture (2003–2018)
Selling_Price	Float	Selling price of the vehicle – the target variable
Present_Price	Float	Current ex-showroom price of the vehicle
Kms_Driven	Integer	Distance driven (500 – 500,000 km)
Fuel_Type	Text	Petrol, Diesel or CNG
Seller_Type	Text	Dealer or Individual
Transmission	Text	Manual or Automatic
Owner	Integer	Number of previous owners (values 0, 1 and 3)

Two exact duplicate rows (an Ertiga and a Fortuner listing) were found and removed, leaving 299 records. The currency unit of Selling_Price and Present_Price is not documented in the supplied materials, so prices are treated as relative values, and all error metrics are expressed in the same unit as Selling_Price.

Table 2: Summary statistics of numerical features (cleaned data, n = 299)

Feature	Mean	Std. dev.	Min	25% / 50% / 75%	Max
Selling_Price	4.5896	4.9842	0.10	0.85 / 3.51 / 6.00	35.00
Present_Price	7.5410	8.5679	0.32	1.20 / 6.10 / 9.84	92.60
Kms_Driven	36,916.8	39,015.2	500	15,000 / 32,000 / 48,883.5	500,000
Year	2013.62	2.90	2003	2012 / 2014 / 2016	2018

The categorical features are imbalanced (Table 3): petrol, dealer-listed and manual-transmission vehicles dominate, and 288 of the 299 records have no previous owner.

Table 3: Categorical feature composition (cleaned data)

Feature	Category counts
Fuel_Type	Petrol 239, Diesel 58, CNG 2
Seller_Type	Dealer 193, Individual 106
Transmission	Manual 260, Automatic 39
Owner	0 owners: 288, 1 owner: 10, 3 owners: 1

Observation made during verification. The Car_Name column shows that the file contains both cars and two-wheelers (for example Royal Enfield, Bajaj, KTM, Hero, Honda Activa and Yamaha models). Classifying the 98 unique names, 198 records are cars and 101 records are two-wheelers. The two groups have very different price ranges: cars have a median Selling_Price of 5.25 (maximum 35.00) while two-wheelers have a median of 0.50 (maximum 1.75). The notebook does not separate the two groups; this is discussed in Section 16.

6. Tools and Technologies

Table 4: Tools and technologies

Tool	Purpose in this project
Python	Programming language for the whole workflow
Pandas, NumPy	Data loading, inspection, cleaning and numerical computation
Matplotlib, Seaborn	Histograms, count plots, scatter plots, boxen plots, heatmap, pair plot and feature-importance chart
Scikit-learn	Train/test split, ColumnTransformer and OneHotEncoder, Pipeline, regression models, RandomizedSearchCV and metrics
Joblib	Saving the final preprocessor and model to best_car_price_model.pkl
Jupyter Notebook	Interactive development and documentation environment

7. Methodology and Workflow

The project follows the nine-step workflow shown in Figure 1. Each step corresponds to a section of the notebook.

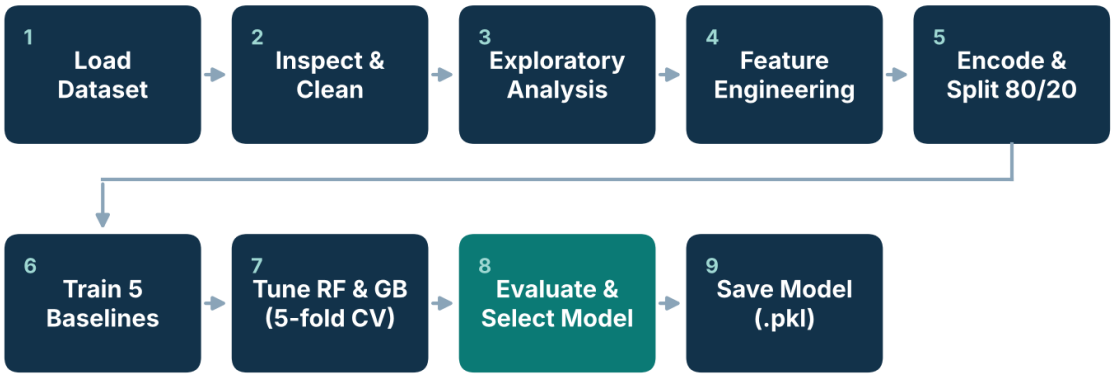


Figure 1: End-to-end workflow of the project (diagram prepared for this report from the notebook's structure)

8. Data Preprocessing

1. **Inspection.** Shape, column names, data types and descriptive statistics were checked. All numerical columns are numeric and the categorical columns are strings.
2. **Missing values and duplicates.** No null values were found. Two duplicate records were detected and removed with drop_duplicates(), reducing the data from 301 to 299 rows.
3. **Feature engineering.** Car_Age = 2018 − Year, using 2018 (the latest year in the data) as the reference year.
4. **Feature selection.** Car_Name was excluded because of its high cardinality (98 unique values for 299 rows), and Year was excluded after Car_Age was derived. The seven selected features are Present_Price, Kms_Driven, Owner, Car_Age, Fuel_Type, Seller_Type and Transmission.
5. **Encoding.** Fuel_Type, Seller_Type and Transmission were one-hot encoded with OneHotEncoder(handle_unknown="ignore") inside a ColumnTransformer, with numerical columns passed through unchanged. This produces 11 model-ready columns.
6. **Train/test split.** The data was split 80/20 with random_state = 42, giving 239 training and 60 test records. The encoder was fitted on the training data only and then applied to the test data.

Outliers (for example a Land Cruiser with Present_Price 92.6 and a scooter with 500,000 km driven) were deliberately kept; no unrecorded outlier transformations were applied. No feature scaling was applied.

9. Exploratory Data Analysis

9.1 Numerical feature distributions

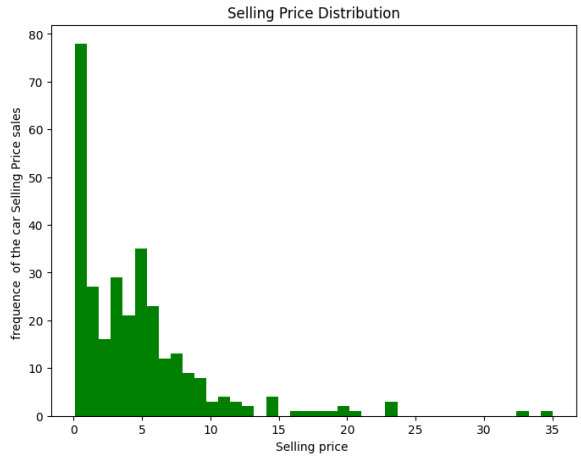


Figure 2: Selling_Price distribution

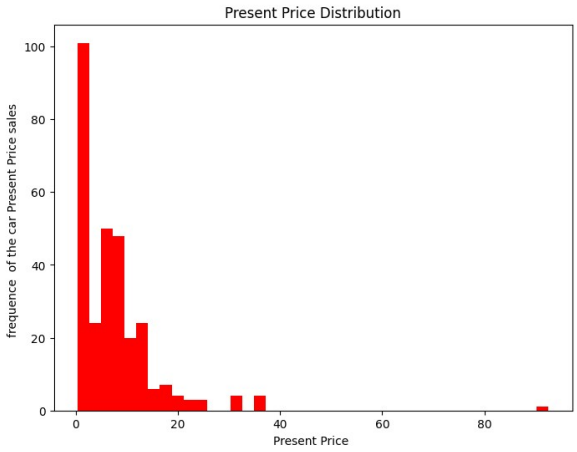


Figure 3: Present_Price distribution

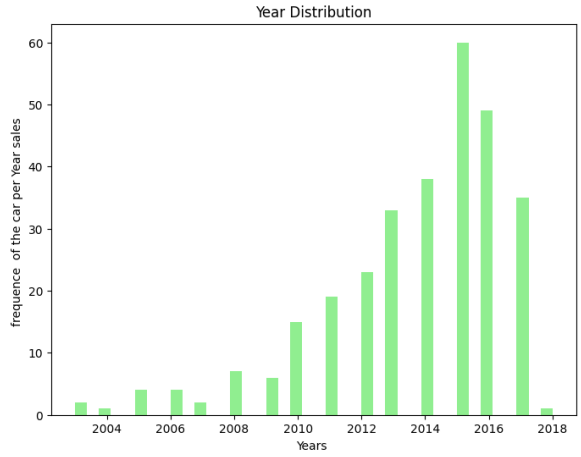


Figure 4: Year distribution

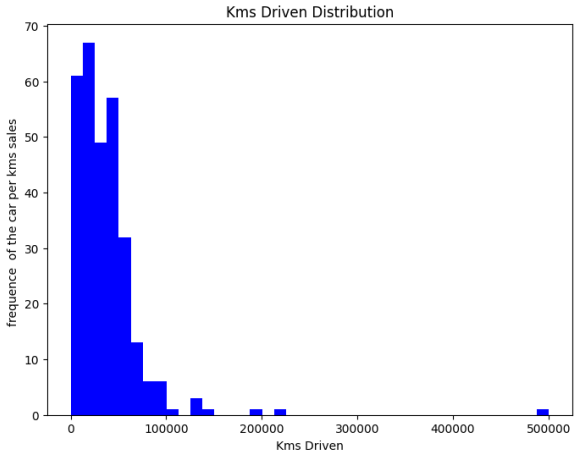


Figure 5: Kms_Driven distribution

Selling_Price and Present_Price are strongly right-skewed: most vehicles are priced low and a few high-value vehicles stretch the range (maximum 35.00 and 92.60). Manufacturing years concentrate between about 2012 and 2017, with the peak around 2015. Kms_Driven is also right-skewed, with most records below 100,000 km and a single extreme value of 500,000 km.

9.2 Categorical feature distributions

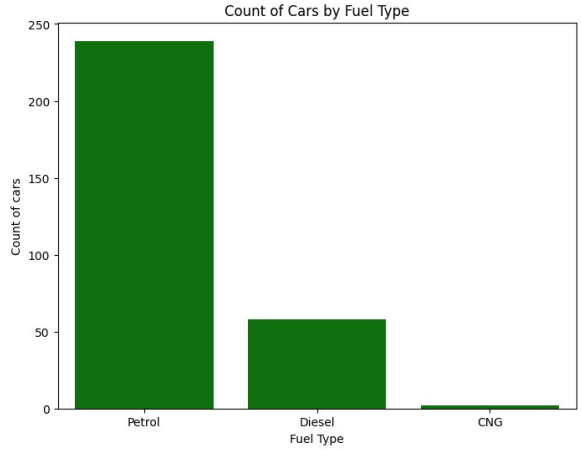


Figure 6: Count of vehicles by Fuel_Type

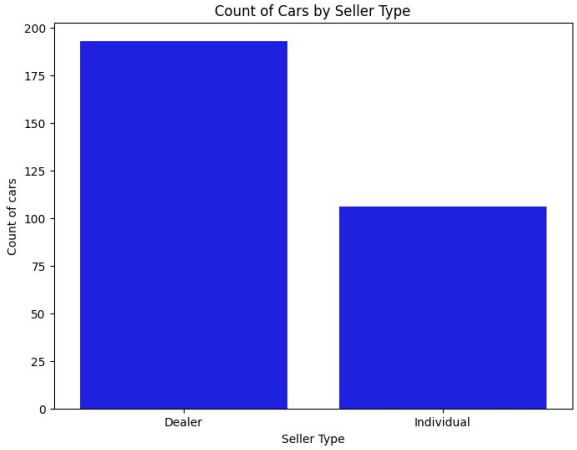


Figure 7: Count of vehicles by Seller_Type

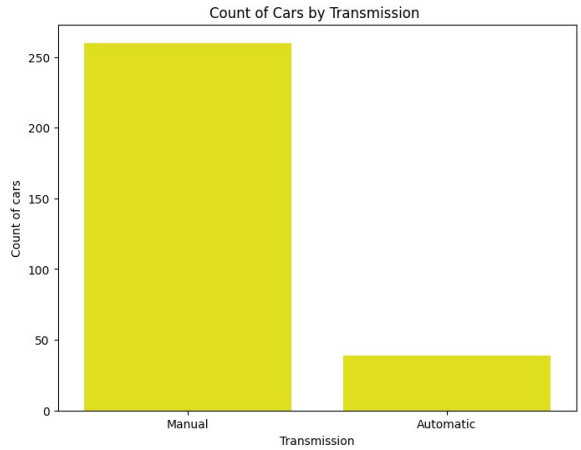


Figure 8: Count of vehicles by Transmission

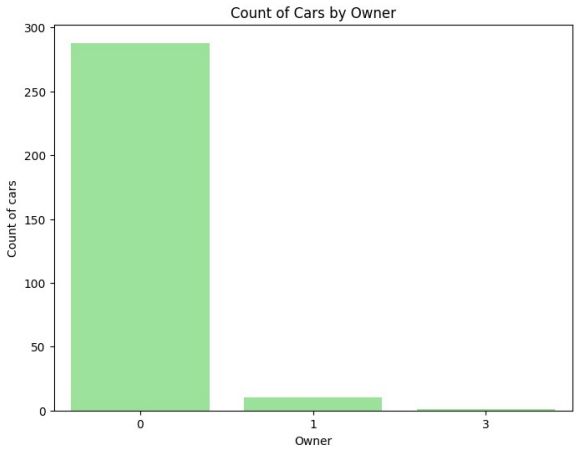


Figure 9: Count of vehicles by Owner

The count plots confirm the class imbalance described in Section 5: petrol (239), dealer (193), manual (260) and zero-owner (288) records are the majority. The CNG category (2 records) and the 3-owner category (1 record) are too small to support reliable conclusions.

9.3 Relationships with Selling_Price



Figure 10: Present_Price vs Selling_Price

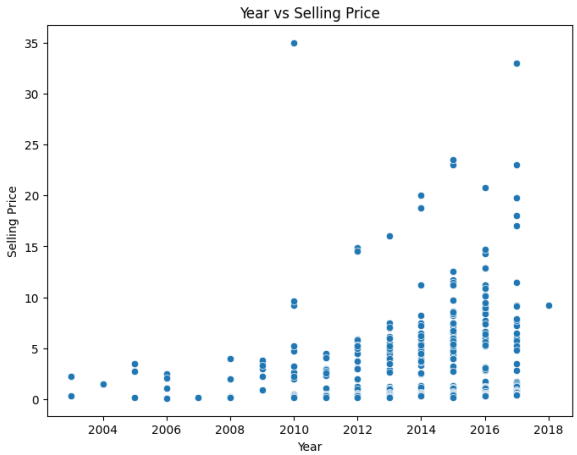


Figure 11: Year vs Selling_Price

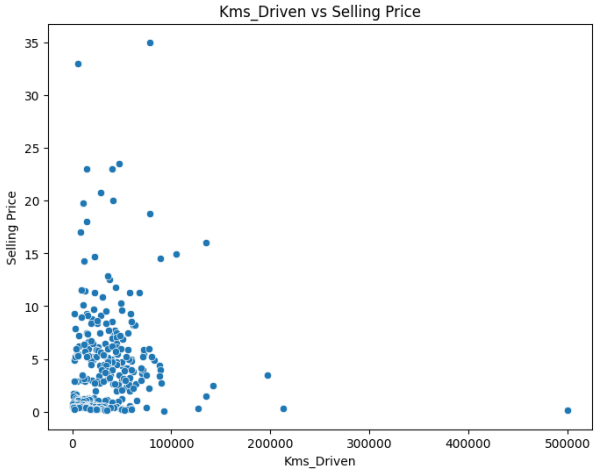


Figure 12: Kms_Driven vs Selling_Price

Present_Price shows a clear, strong, roughly linear upward relationship with Selling_Price, which is the clearest pattern in the data. Newer vehicles tend to sell for more, but the relationship with Year is much more scattered. Kms_Driven shows no clear linear trend in the scatter plot; most high selling prices occur at low-to-moderate mileage, and the single 500,000 km record sits far from the rest.

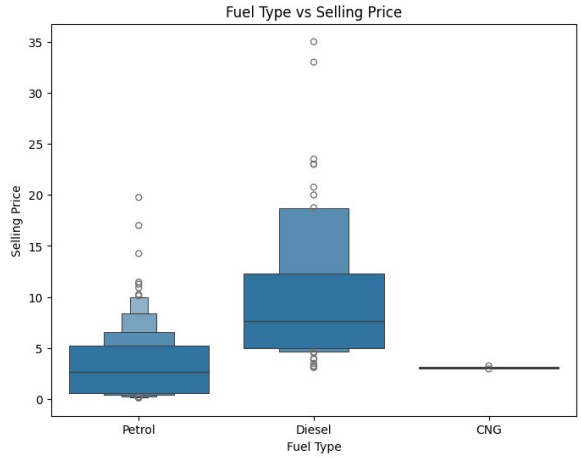


Figure 13: Fuel_Type vs Selling_Price

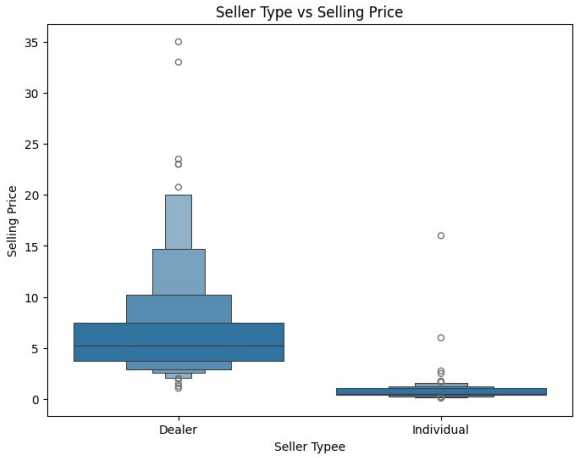


Figure 14: Seller_Type vs Selling_Price

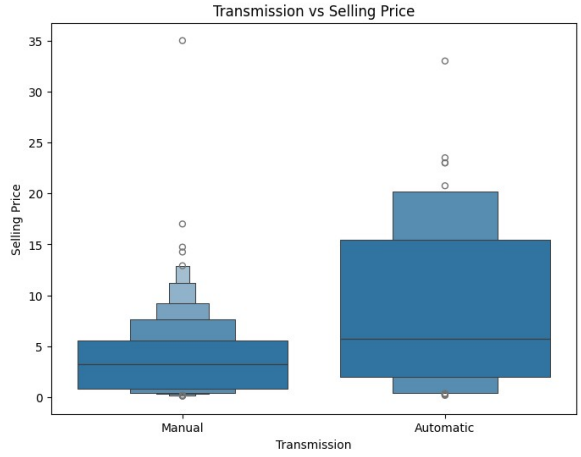


Figure 15: Transmission vs Selling_Price

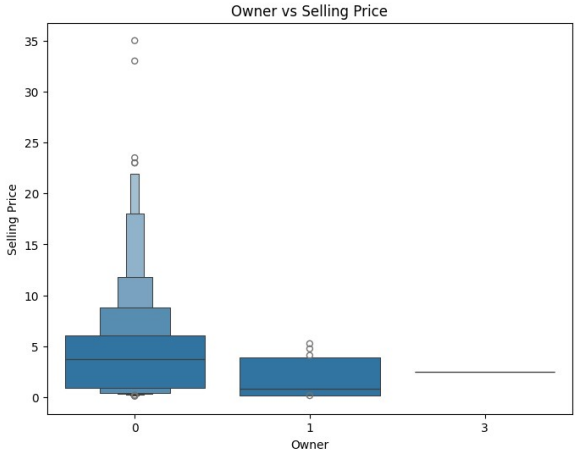


Figure 16: Owner vs Selling_Price

In the boxen plots, diesel vehicles, dealer listings and automatic-transmission vehicles show higher selling-price distributions than petrol vehicles, individual sellers and manual-transmission vehicles. Median Selling_Price by group, computed from the cleaned data, is: Dealer 5.25 vs Individual 0.52;

Automatic 5.75 vs Manual 3.25; Diesel 7.60, CNG 3.10 and Petrol 2.65. These group differences are partly influenced by the vehicle mix in the dataset (see Section 16), and the CNG and multi-owner groups contain very few records.

9.4 Correlation analysis



Figure 17: Correlation heatmap of the numerical variables

The heatmap confirms that Present_Price has by far the strongest linear association with Selling_Price (r = 0.88). Year has a weak positive correlation (0.23), Owner a weak negative correlation (−0.088) and Kms_Driven essentially none (0.029). Year and Kms_Driven are moderately negatively correlated (−0.53), which is expected because newer vehicles have usually been driven less.

Note on the notebook text. The notebook's heatmap commentary describes Kms_Driven as negatively associated with Selling_Price. The heatmap itself shows a coefficient of +0.029, so this report follows the plotted values: the linear association between Kms_Driven and Selling_Price is negligible in this dataset, likely because the large outlier and the mix of vehicle types mask any mileage effect.

9.5 Pair plot

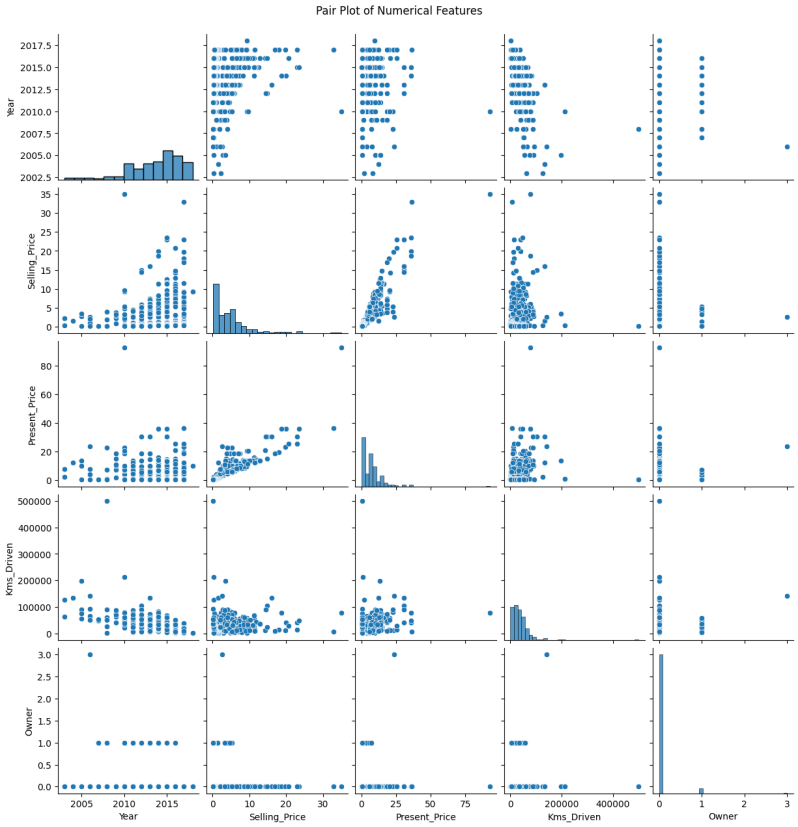


Figure 18: Pair plot of Year, Selling_Price, Present_Price, Kms_Driven and Owner

The pair plot gives a combined view of all pairwise relationships and marginal distributions. It reinforces the positive Present_Price–Selling_Price relationship and makes the isolated high-value and high-mileage observations visible.

10. Model Development and Implementation

Five baseline regression models were trained on the encoded training set (239 records) and evaluated on the held-out test set (60 records). Random states were fixed (random_state = 42) for the tree-based models so that results are reproducible; all other hyperparameters were left at the scikit-learn defaults.

Table 5: Baseline regression models

Model	Description
Linear Regression	Fits a linear combination of the features; a simple, interpretable baseline
Decision Tree Regressor	Splits the feature space into regions with a single predicted value per leaf
Random Forest Regressor	Averages many decision trees trained on bootstrap samples
Gradient Boosting Regressor	Builds trees sequentially, each correcting the errors of the previous ensemble
Support Vector Regressor (SVR)	Kernel-based regression; default settings, without feature scaling

Performance was measured with four metrics: MAE (average absolute error), MSE (average squared error, which penalises large errors more strongly), RMSE (square root of MSE, in the units of the target) and R^2 (proportion of variance explained; a negative value means the model is worse than always predicting the mean).

11. Hyperparameter Tuning

RandomizedSearchCV was applied to the Random Forest and Gradient Boosting models, each wrapped in a Pipeline together with the preprocessor so that encoding is refitted inside every cross-validation fold. For each model, 50 random combinations were sampled and evaluated with five-fold cross-validation on the training data (250 fits per model), using negative RMSE as the scoring objective.

Table 6: Hyperparameter search spaces

Hyperparameter	Random Forest search space	Gradient Boosting search space
n_estimators	100, 200, 300, 400, 500	50, 100, 150, 200, 300
max_depth	None, 5, 10, 15, 20, 30	2, 3, 4, 5
min_samples_split	2, 5, 10, 15	2, 5, 10
min_samples_leaf	1, 2, 4, 6	1, 2, 4
max_features	sqrt, log2, 1.0	–
learning_rate	–	0.01, 0.03, 0.05, 0.1, 0.2
subsample	–	0.7, 0.8, 0.9, 1.0

Table 7: Best hyperparameters found by RandomizedSearchCV

Selected value	Random Forest	Gradient Boosting
n_estimators	100	300
max_depth	20	4
min_samples_split	2	5
min_samples_leaf	2	4
max_features	1.0	–
learning_rate	–	0.1
subsample	–	0.8
Best cross-validation RMSE	1.5436	1.3817

Both tuned models achieved a cross-validation RMSE (1.54 and 1.38) that is lower than their RMSE on the test split (3.27 and 3.07). This gap is not unusual for a small dataset: the cross-validation estimate averages over folds drawn from the training data, whereas the 60-record test split may contain a few high-priced vehicles that are harder to predict.

12. Model Evaluation and Results

Table 8: Test-set results for all seven models (sorted by R^2)

Model	MAE	MSE	RMSE	R^2 Score
Gradient Boosting (Baseline)	**1.1349**	**6.3667**	**2.5232**	**0.7530**
Linear Regression	1.4729	6.3708	2.5240	0.7528
Decision Tree	1.3273	8.4855	2.9130	0.6708
Gradient Boosting (Tuned)	1.3016	9.4343	3.0715	0.6339
Random Forest (Tuned)	1.3851	10.6659	3.2659	0.5862
Random Forest (Baseline)	1.5270	13.7505	3.7082	0.4665
SVR (Baseline)	3.1295	28.3205	5.3217	−0.0988

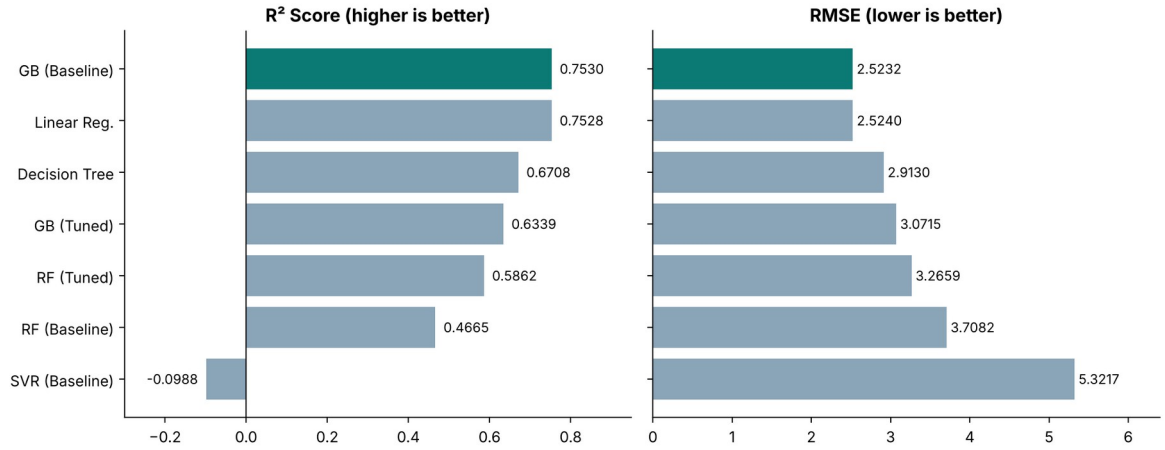


Figure 19: R^2 and RMSE of all seven models on the test set (chart prepared from the verified results in Table 8)

The baseline Gradient Boosting Regressor achieved the highest R^2 (0.7530), the lowest RMSE (2.5232) and the lowest MAE (1.1349). Its advantage over Linear Regression is very small on R^2 and RMSE (0.7530 vs 0.7528 and 2.5232 vs 2.5240); the clearer difference is in MAE, where Gradient Boosting's average error is about 23% lower (1.1349 vs 1.4729). With only 60 test records, the first two models should be regarded as performing similarly on squared-error metrics.

Tuning improved Random Forest (R^2 from 0.4665 to 0.5862, RMSE from 3.7082 to 3.2659) but reduced Gradient Boosting's test performance (R^2 from 0.7530 to 0.6339). The tuned models therefore did not outperform the baseline Gradient Boosting model on the test split. SVR performed worst, with a negative R^2 (−0.0988), meaning it predicted worse than the mean of the target on this test set.

Note on the notebook text. The notebook's tuning commentary states that tuning did not improve the test-set performance of the baseline versions. The recorded results show that this holds for Gradient Boosting but not for Random Forest, whose tuned version improved on its baseline; this report states the results as recorded.

13. Feature Importance Analysis

Feature importance was extracted from a baseline Gradient Boosting model fitted on the encoded training data (Figure 20, Table 9).

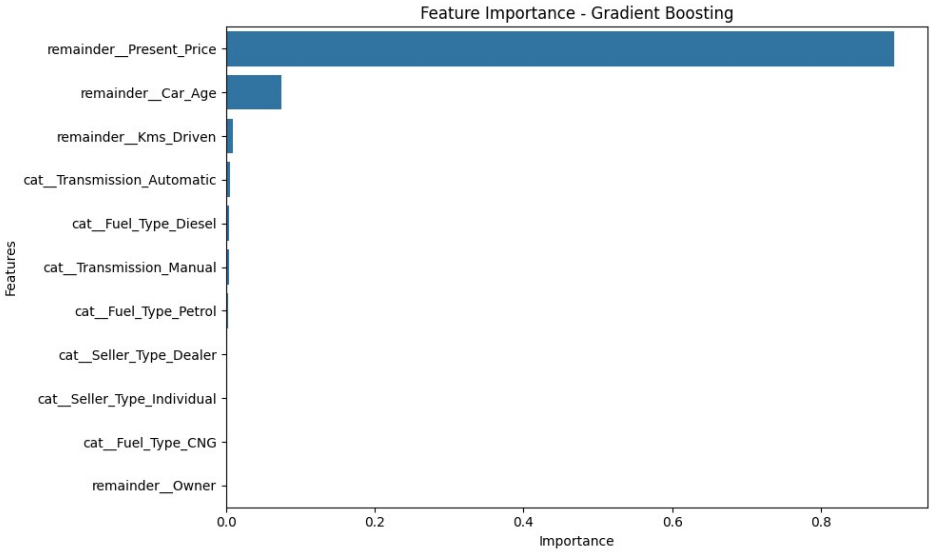


Figure 20: Feature importance of the baseline Gradient Boosting model

Table 9: Feature importances (baseline Gradient Boosting)

Feature	Importance
Present_Price	0.898545
Car_Age	0.074910
Kms_Driven	0.009688
Transmission_Automatic	0.005286
Fuel_Type_Diesel	0.004270
Transmission_Manual	0.003892
Fuel_Type_Petrol	0.003307
Seller_Type_Dealer	0.000071
Seller_Type_Individual	0.000025
Fuel_Type_CNG	0.000004
Owner	0.000002

Present_Price dominates (about 89.85%), followed by Car_Age (about 7.49%) and Kms_Driven (about 0.97%). This is consistent with the EDA: the current price of a vehicle is a strong signal of its resale value, and age captures depreciation. The remaining encoded categorical features and Owner contribute very little to this fitted model.

14. Challenges Faced and Solutions

Table 10: Challenges and solutions

Challenge	How it was handled
High-cardinality Car_Name (98 unique names for 299 rows)	Excluded from the model inputs; one-hot encoding it would have produced many sparse columns for a very small dataset.
Year is a calendar value, not a measure of age	Engineered Car_Age = 2018 − Year, and removed Year to avoid redundancy.
Duplicate records	Two exact duplicates were identified and removed before analysis.

Challenge	How it was handled
Skewed distributions and extreme values	Kept in the data for analysis and modelling; tree-based models are not sensitive to feature scale, and no unrecorded transformations were applied.
Categorical features unsuitable for direct use	Applied one-hot encoding inside a ColumnTransformer, fitted on training data only to avoid information leakage.
Tuning inside cross-validation	The preprocessor was placed inside a Pipeline, so each fold is encoded separately and the validation fold stays unseen.
Tuned models did not beat the baseline on the test split	Reported honestly and the baseline Gradient Boosting model was retained; the report explains the likely role of the small dataset and a single test split.

15. Results and Discussion

The best model, the baseline Gradient Boosting Regressor, explains about 75% of the variance in selling prices on the test set (R^2 0.7530) and has an average absolute error of 1.1349 price units. For context, the test-set Selling_Price has a standard deviation of about 5.12, so an RMSE of 2.52 is roughly half of the natural spread of the target.

Three observations stand out. First, model complexity did not guarantee better results: the simple Linear Regression baseline matched Gradient Boosting on squared-error metrics, which is consistent with the strong, almost linear dependence of Selling_Price on Present_Price. Second, tuning gave mixed results, which shows the sensitivity of small-data evaluation to the particular split. Third, almost all of the predictive power comes from a single feature, Present_Price, with Car_Age adding a smaller but meaningful contribution.

The selected model should be regarded as a solid exploratory baseline rather than a production-ready price estimator, for the reasons listed in the next section.

16. Limitations

- The dataset is small (299 records after duplicate removal), so results can change noticeably with a different train/test split.
- Baseline model comparison and selection used the test-set results. The reported test performance is therefore exploratory and not an entirely unbiased estimate of final performance.
- The baseline SVR was trained without feature scaling, although SVR is sensitive to feature scale. Its poor result (negative R^2) should not be read as a fair assessment of the algorithm.
- The dataset may not represent all used-vehicle markets or current market conditions; it covers manufacturing years 2003–2018 only.
- The file contains cars (198 records) and two-wheelers (101 records) with very different price ranges, and the models were not told which is which. Group effects such as the large gap between dealer and individual sellers may partly reflect this mix rather than a pure seller effect.

- The categorical groups CNG (2 records) and Owner = 3 (1 record) are too small for reliable conclusions, and one record has 500,000 km driven.
- The currency unit of the price columns is not documented, so error values are interpretable only relative to the price scale of the data.
- Gradient Boosting's margin over Linear Regression on R^2 and RMSE is very small; with 60 test records, the difference is not strong evidence that one model is better.

17. Future Improvements

- Select models using cross-validation on the training data (or a separate validation set) and keep the test set for one final evaluation.
- Train and evaluate with feature scaling for SVR (for example with StandardScaler in a Pipeline) and compare again.
- Model cars and two-wheelers separately, or add a vehicle-type feature, and test whether performance improves.
- Try modelling log(Selling_Price) to reduce the effect of the right-skewed target, and examine residuals by price range.
- Use repeated cross-validation to obtain confidence intervals for each model's metrics.
- Collect a larger and more recent dataset covering more markets.
- Wrap the saved model in a simple application (for example a web form or API) so that prices can be predicted interactively.

18. Conclusion

This project built a complete regression workflow for predicting used-vehicle selling prices. After cleaning the data and engineering Car_Age, five baseline models and two tuned ensemble models were compared on a held-out test set. The baseline Gradient Boosting Regressor performed best (MAE 1.1349, RMSE 2.5232, R^2 0.7530), and Present_Price was by far the most influential feature (about 89.85%), followed by Car_Age (about 7.49%). The results are encouraging for a small dataset, and the limitations, mainly the small sample, the exploratory use of the test set and the mixed vehicle types, point to clear next steps for improving reliability.

19. Reproducibility and Saved Model

The final model was saved with Joblib as best_car_price_model.pkl. The file stores a dictionary with two entries: the fitted preprocessor (ColumnTransformer with OneHotEncoder) and the fitted baseline Gradient Boosting model. To predict, new data with the seven input columns is first transformed with the preprocessor and then passed to the model. During the preparation of this report the file was loaded and used to predict the test split; it reproduced R^2 = 0.7530 and MAE = 1.1349, matching the notebook.

All baseline and tuned results in this report were also reproduced independently from car.csv using the notebook's settings (random_state = 42, 80/20 split), and matched the notebook's reported values. To rerun the project, install the dependencies (pip install pandas numpy matplotlib seaborn

scikit-learn jupyter joblib), place car.csv in the data folder expected by the notebook (the notebook reads ../data/car.csv) and run the notebook cells in order.

References

[1] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

[2] Friedman, J. H. (2001). Greedy function approximation: a gradient boosting machine. Annals of Statistics, 29(5), 1189–1232.

[3] Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5–32.

[4] Bergstra, J. and Bengio, Y. (2012). Random search for hyper-parameter optimization. Journal of Machine Learning Research, 13, 281–305.

[5] McKinney, W. (2010). Data structures for statistical computing in Python. Proceedings of the 9th Python in Science Conference, 56–61.

[6] Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. Computing in Science & Engineering, 9(3), 90–95.

[7] Waskom, M. L. (2021). seaborn: statistical data visualization. Journal of Open Source Software, 6(60), 3021.

[8] Scikit-learn documentation: GradientBoostingRegressor, RandomizedSearchCV, ColumnTransformer and OneHotEncoder. https://scikit-learn.org/stable/